



Optimización y Mejoras del Pipeline ETL

Reto 3 — Diseño y Optimización de Flujos de Trabajo ETL Avanzados

Autor: Judit Giravent

Herramienta: Apache NiFi 1.28.1

Fecha: Septiembre 2026

1. Introducción

Este documento recoge las **lecciones aprendidas** durante la implementación del pipeline ETL, los **problemas encontrados** y las **mejoras aplicables** para optimizar el rendimiento, la robustez y la mantenibilidad del flujo. Todos los consejos están **basados en problemas reales** experimentados durante el desarrollo del Reto 3.

2. Mejoras aplicadas durante el desarrollo

2.1 Problema: CoinGecko rechazaba las peticiones de NiFi (Error 403)

Síntoma: El `InvokeHTTP` devolvía `403 Forbidden` con el mensaje:

```
Please add a descriptive User-Agent to your request.
```

Causa: NiFi 1.28.1 envía un `User-Agent` por defecto (`NiFi/1.28.1`) que CoinGecko rechaza. Aunque configuramos `UpdateAttribute` y `Request Header Attributes Pattern`, **no fue posible sobrescribir el `User-Agent`** desde la interfaz.

Solución aplicada: Descargar el JSON previamente con PowerShell y usar `GetFile` como origen.

Mejor solución a largo plazo:

- **Configurar el `User-Agent` a nivel de JVM** en NiFi, editando `conf/bootstrap.conf` con:

```
java.arg.20=-Dhttp.agent=OdiseaData-NiFi/1.0
```

- **O usar un `InvokeHTTP` con un proxy intermedio** (como un script Python + FastAPI) que sí gestione las cabeceras.
- **O usar una API que no requiera `User-Agent`** como `open.er-api.com` o `api.exchangerate-api.com`.

2.2 Problema: El procesador `ExecuteSQLRecord` no se ejecutaba

Síntoma: Contadores en `In: 0, Out: 0`, sin errores visibles.

Causa: Los **procesadores de origen** (`ExecuteSQLRecord`, `InvokeHTTP`) **no se ejecutan solos**, necesitan un `FlowFile` de entrada que los dispare.

Solución aplicada: Añadir un `GenerateFlowFile` con `Run Schedule = 1 day` antes.

Mejor solución a largo plazo:

- Usar `GenerateFlowFile` con schedule CRON más específico para automatizar (ej: `0 0 2 * * ?` = todos los días a las 2 AM).
- Añadir un `RouteOnAttribute` para hacer un control de calidad previo.

2.3 Problema: `MergeContent` no mostraba las propiedades `Minimum/Maximum Number of Entries`

Síntoma: Las propiedades no aparecían en la pestaña PROPERTIES.

Causa: En NiFi 1.28.1, las propiedades visibles del `MergeContent` **cambian según el** `Merge Strategy` seleccionado. La estrategia `Defragment` oculta las propiedades de control de tamaño.

Solución aplicada: Cambiar `Merge Strategy` a `Bin-Packing Algorithm`.

Mejor solución a largo plazo:

- Usar `Merge Strategy = Defragment` **solo cuando los FlowFiles ya tengan atributos de fragmento** (`fragment.identifier`, `fragment.count`).
- Para unir FlowFiles independientes, usar siempre `Bin-Packing Algorithm`.

2.4 Problema: `PutFile` daba `InvalidPathException`

Síntoma:

```
java.nio.file.InvalidPathException: Illegal char < > at index 35
```

Causa: El nombre del archivo contenía **caracteres invisibles** o `:` introducidos al copiar/pegar la expresión `${now():format('yyyyMMdd_HH:mm:ss')}`.

Solución aplicada: Reescribir la propiedad `filename` a mano con un formato sin `:` (`dataset_final.json`).

Mejor solución a largo plazo:

- **Formato de timestamp recomendado:** `yyyyMMddHHmmss` (todo pegado, sin separadores).
- **Validar los nombres de archivo** con un `ValidateRecord` antes del `PutFile`.
- **Usar el procesador** `ReplaceText` para limpiar caracteres inválidos:
 - Search Value: `[ : * ? " < > | ]`
 - Replacement Value: `_`

3. Mejoras de rendimiento (optimización)

3.1 Paralelización de procesadores

Configuración actual: Todos los procesadores tienen `Concurrent Tasks = 1`.

Mejora recomendada:

- En procesadores **CPU-intensivos** (`ConvertRecord`, `ExecuteSQLRecord`), aumentar a **2-4 tareas concurrentes**.

- En procesadores **I/O-intensivos** (`GetFile`, `PutFile`), mantener en **1-2 tareas**.
- **Cuidado:** no paralelizar sin control, puede provocar problemas de concurrencia en la escritura a disco.

Configuración:

Settings → Concurrent Tasks = 4

3.2 Ajuste del `Run Schedule`

Configuración actual: 1 day en todos los procesadores de origen.

Mejora recomendada según el caso:

Fuente	Frecuencia recomendada	Justificación
CSV ventas	0 0 3 * * ? (diario a las 3 AM)	Bajo volumen de cambios
API cripto	0 0 * * * ? (cada hora)	Precios volátiles
SQLite clientes	0 0 4 * * ? (diario a las 4 AM)	Datos maestros
Merge final	Ejecución tras las 3 ramas	Necesita las 3 fuentes

3.3 Uso de `MergeContent` con límites

Configuración actual: `Minimum = 3`, `Maximum = 3`.

Mejora recomendada:

- Añadir `Maximum Bin Age = 5 min` para evitar que el `MergeContent` espere indefinidamente si alguna rama falla.
- Configurar `Minimum Group Size` y `Maximum Group Size` en bytes para manejar datasets grandes.
- Activar `Compression` (GZIP) si el tamaño final es grande:

Compression Format: GZIP

3.4 Uso de almacenamiento temporal para volúmenes grandes

Configuración actual: Todo el flujo opera en memoria.

Mejora recomendada para volúmenes > 1 GB:

- Usar `PutFile` intermedio entre fases (staging).
- Activar `Swap Files` en el `FlowFile Repository` de NiFi (ya está por defecto).
- Configurar `nifi.content.claim.max.appendable.size` en `nifi.properties` para archivos grandes.

3.5 Caché de datos en NiFi

Configuración actual: Sin caché.

Mejora recomendada:

- Usar `DistributedMapCacheClientService` para almacenar resultados intermedios.
- Configurar `LookupRecord` con `SimpleCsvFileLookupService` para enriquecer datos sin volver a llamar a la API.

Ejemplo:

- Si el mismo JSON de cripto se usa varias veces, guardarlo en caché.
- Evitar llamadas repetidas a APIs externas.

4. Mejoras de robustez

4.1 Añadir control de errores en cada rama

Configuración actual: Algunas relaciones `failure` no están conectadas a nada.

Mejora recomendada:

- Conectar **todas las relaciones** `failure` a un `LogAttribute` común.
- Añadir `RouteOnAttribute` para clasificar errores por tipo.
- Enviar notificaciones con `PutEmail` o `PutSlack` cuando haya fallos.

Ejemplo:

```
PutFile → failure → LogAttribute → RouteOnAttribute → PutEmail (alerta)
```

4.2 Validación con `ValidateRecord`

Configuración actual: Sin validación formal.

Mejora recomendada:

- Añadir `ValidateRecord` antes de cada `PutFile` para verificar:
 - Tipos de datos correctos.
 - Campos obligatorios presentes.
 - Sin valores nulos inesperados.

Ejemplo:

```
ConvertRecord → ValidateRecord → PutFile
```

4.3 Deduplicación con `DetectDuplicate`

Configuración actual: Sin deduplicación.

Mejora recomendada:

- Añadir `DetectDuplicate` para evitar procesar el mismo CSV dos veces.
- Usar un **atributo clave único** (por ejemplo, `invoice_id`).

4.4 Reintentos automáticos

Configuración actual: Sin reintentos.

Mejora recomendada:

- Activar `Retry` en `InvokeHTTP` para reintentar peticiones fallidas.
- Configurar `Retry FlowFile` en procesadores que fallen.

Ejemplo:

InvokeHTTP → Retry → RetryFlowFile (con backoff exponencial)

5. Mejoras de monitorización

5.1 Reportes automáticos con `PrometheusReportingTask`

Mejora recomendada:

- Activar `PrometheusReportingTask` para exponer métricas a un servidor Prometheus.
- Visualizar con **Grafana**: throughput, FlowFiles en cola, tiempo de procesamiento.

5.2 Alertas con `MonitorActivity`

Mejora recomendada:

- Añadir `MonitorActivity` para detectar cuándo un procesador lleva mucho tiempo sin procesar.
- Enviar alerta si el flujo lleva más de X minutos sin actividad.

5.3 Logs centralizados

Mejora recomendada:

- Configurar NiFi para enviar logs a **Elasticsearch** o **Splunk**.
- Añadir `LogAttribute` con nivel `WARN` en procesadores críticos.

5.4 Dashboard de estado en NiFi

Mejora recomendada:

- Configurar el **NiFi Summary** con las métricas clave.
- Añadir **Bulletins** con filtros por procesador.
- Usar **Controller Service Status** para monitorizar los servicios activos.

6. Mejoras de mantenibilidad

6.1 Versionado con NiFi Registry

Mejora recomendada:

- Configurar **NiFi Registry** para versionar los flujos.
- Cada cambio importante se registra como una nueva versión.
- Facilita el rollback en caso de errores.

6.2 Plantillas reutilizables

Mejora recomendada:

- Crear **plantillas de NiFi** para cada tipo de fuente (CSV, API, SQLite).
- Reutilizarlas en futuros proyectos.
- Exportar como `.xml` y guardar en el repo.

6.3 Documentación automática

Mejora recomendada:

- Añadir **Comments** en cada procesador explicando su función.
- Usar `updateAttribute` para añadir metadatos a los FlowFiles (`source`, `timestamp`, `version`).
- Generar documentación del flujo con la herramienta `nifi-toolkit`.

7. Optimización del consumo de recursos

7.1 Ajuste de memoria JVM

Configuración actual: `xmx1g` (por defecto).

Mejora recomendada según carga:

Volumen de datos	Memoria recomendada
< 1 GB	<code>Xmx2g</code>
1-10 GB	<code>Xmx4g</code>
> 10 GB	<code>Xmx8g</code> o más

Configuración en `conf/bootstrap.conf`:

```
java.arg.2=-Xms2g
java.arg.3=-Xmx4g
```

7.2 Ajuste del tamaño de colas

Mejora recomendada:

- Limitar el tamaño de las colas para evitar acumulación:

```
Settings → Back Pressure Object Threshold = 10000
Settings → Back Pressure Data Size Threshold = 1 GB
```

- Configurar `FlowFile Expiration` para que los FlowFiles caduquen si no se procesan.

7.3 Uso de SSD

Mejora recomendada:

- Mover `content_repository`, `flowfile_repository` y `provenance_repository` a un **disco SSD**.
- Evitar discos mecánicos para mejorar el throughput.

8. Mejoras específicas para este proyecto

8.1 Reducir el tamaño del CSV de entrada

Situación actual: Se usan 10.000 filas de las 541.909 originales.

Mejora recomendada:

- **Dividir el CSV en lotes** con `SplitRecord`:

```
Records Per Split = 10000
```

- Procesar cada lote en paralelo con múltiples hilos.

8.2 Añadir normalización y enriquecimiento

Situación actual: Los datos se cargan "tal cual".

Mejora recomendada:

- Añadir `UpdateRecord` para:
 - Normalizar fechas al formato ISO 8601.
 - Calcular campos derivados (ej: `precio_total = precio_unitario * cantidad`).
 - Unificar monedas (ej: convertir todos los precios a EUR).
- Añadir `LookupRecord` para enriquecer con datos externos.

8.3 Cargar a un data warehouse real

Situación actual: Se guarda en JSON estático.

Mejora recomendada:

- Cargar en **PostgreSQL** o **BigQuery** usando `PutDatabaseRecord`.
- Usar `PutParquet` para almacenar en formato columnar (más eficiente).

8.4 Integración con un dashboard

Situación actual: Dataset final en JSON.

Mejora recomendada:

- Cargar los datos a **Looker Studio** o **Power BI**.
 - Crear un dashboard con KPIs del pipeline:
 - Throughput diario.
 - Errores por fuente.
 - Tiempo medio de procesamiento.
-

9. Conclusión

El pipeline ETL desarrollado es **funcional, reproducible y extensible**. Las mejoras propuestas permiten:

- **Escalar** el flujo a volúmenes mayores (paralelización, SSD, memoria).
- **Robustecer** el sistema contra errores (validación, reintentos, alertas).
- **Mantener** el flujo a largo plazo (NiFi Registry, documentación).
- **Optimizar** el consumo de recursos (caché, back pressure).

La aplicación de estas mejoras convertiría el pipeline en un sistema **listo para producción**, capaz de procesar grandes volúmenes de datos de forma continua y confiable.